MACHINA PARTY

プログラム説明資料

2026年11月 更新版



ジャンル：マルチプレイ 3D アクション / パーティーゲーム

開発環境：C# / Unity6　|　開発期間：2026/4 〜 制作中（2人制作）

横浜デジタルアーツ専門学校 ゲーム科 3年　石川原 吉美

担当：ゲーム内容全般

C# | Unity6 | NavMesh | CharacterController | Input System

開発体制・使用ツールについて

01 所要時間の概算

- 開発開始から現在まで約7か月（2026年4月〜）
- 週3〜4回×2〜3時間ペースで継続中
- 自身の実装時間目安　**約300時間**（11月時点）

02 開発体制・担当範囲

- 2人体制（自身＋1名）
- 自身の担当：ゲーム内容全般
- 開発環境：Unity6 / C#

03 学校のFW・AI活用範囲

- 学校指定の共通フレームワークは不使用、ゼロから構築
- 実装のエラー・不明点はAIに質問し参考にした
- タイトル画面イラストはAI画像生成（ChatGPT）で制作
- UI系統もAIに生成させて使用
- ゲームロジックの設計・実装は自身で実施

04 外部アセット・ミドルウェア

- AI生成以外のグラフィックはフリー素材を使用し、Photoshopで加工
- ミドルウェア・外部SDKの組み込みはなし

アピールポイント①　タンクモード

操作・射撃・AI・カメラを実装

01 GameManager 設計

ラウンド制御・生存判定・勝者判定を1クラスに集約。PlayerDataでCPU/Human・カラー・操作設定をメニューからゲームへ受け渡す。

02 TankMovement（移動）

CharacterControllerで3D移動。キーボード／ダイレクト／ゲームパッドの3操作方式に対応し、爆風ノックバックも加速度減衰で実装。

03 TankShooting（射撃）

ボタン長押しでチャージし、離して発射（CPUは最大で自動発射）。放物線計算でAIの着弾予測地点を算出。強化弾システムも搭載。

04 TankAI（CPU制御）

NavMeshで最短経路を計算し敵を追跡(Seek)。近づきすぎると逃走(Flee)に遷移。射程・壁チェックで射撃タイミングを自律判定。

05 TankHealth（HP管理）

HPバーをLerpでアニメーション。被弾後の無敵時間(点滅演出)、シールドによるダメージ軽減、ノックバック無効化と連携。

06 カメラ演出

CameraControlが全員を画面に収めるよう追従。TankOcclusionFadeが、視界を遮る岩を半透明にして自機の見失いを防ぐ。

アピールポイント②　レースモード

Mario Kart風のレースを、進行・走行・ロジック・表示に分離して実装

01 RaceGameManager（進行）

車種選択→スタート演出→レース→リザルトをフェーズで一元管理。参加者はRaceSessionで引き継ぎ、リトライ・車種選択・タイトルへ戻れる。

02 CarMovement（走行）

速度を直接制御するマリカー式の走行。ドリフトで溜めたゲージが3段階のミニターボに。坂は地面の法線に合わせて車体を傾ける。

03 車タイプ（CarSpec）

5項目を1〜5点で配分し、合計は常に15点。5車種を用意し、点数が最高速・加速・旋回・滑りやすさ・重さに反映される。

04 周回・順位ロジック

周回・順位・終了判定をUnity非依存の純C#に分離。チェックポイントを順番に通った時だけ周回を数え、逆走・飛ばしを防止。

05 画面分割とミニマップ

1〜4人の分割画面を自動構成。HUD・ミニマップ・リザルトのUIはAIに生成させたものを使用。コース形状はRayで自動検出。

06 入力・デバイス対応

InputReaderがキーボード2系統とゲームパッド複数台を吸収。ControlIndexで人ごとに割り当て、ゴール後はInputLockedで操作停止。

ゲームシステム — アーキテクチャ全体図

3つのシーンと、各シーンを構成する主なクラス

Main Menu

GameUIHandler
メニュー・参加スロット・ポーズ管理

StartMenuSlot
プレイヤー設定UI

PauseMenu
ポーズ画面制御

TitleScreenButtonInput
キー／パッドでタイトルのボタン操作

RaceSceneLoader
レースシーンへの遷移

TankMode Scene

GameManager
ラウンド進行・生存判定

TankManager
タンク個別管理

TankMovement
CharacterController移動

TankShooting
チャージ射撃

TankAI
NavMesh CPU AI

TankHealth
HP・無敵・シールド

CameraControl
全体追従カメラ＋遮蔽物フェード

RaceMode Scene

RaceGameManager
レース進行・フェーズ管理

CarMovement / InputReader
走行・ドリフト／入力

LapCounter・CheckPoint・GoalTrigger
周回・通過判定

RaceRules / CarSpec / RaceLayout
順位・車性能・配置（純C#）

RaceSplitScreenCameraManager
分割画面・追従カメラ

RaceHud / RaceMinimap / RaceResultsUI
HUD・ミニマップ・結果

CarSelectMenu / RaceStartSequence
車種選択・スタート演出

PlayerData：メニュー設定→GameManager ｜ RaceSession：レース参加者をシーン間で保持 ｜ InputReader：全モード共通の入力層

GameManager 設計

ゲーム進行管理 — ラウンド制御・生存判定・タンク生成

ラウンド制御

GameLoop()コルーチンがRoundStarting→RoundPlaying→RoundEndingを順に処理。m_NumRoundsToWin(5)勝した時点でLoadScene(0)により終了。

生存判定

OneTankLeft()でactiveSelfなタンク数を毎フレーム計測。1体以下でRoundEndingへ進む。全滅時は「DRAW!」を表示。

PlayerData

IsComputer・TankColor・UsedPrefab・ControlIndexをメニューからGameManagerに渡す内部クラス。人数分の配列でまとめて受け取る。

SpawnAllTanks()

PlayerData配列の人数分Instantiateし、全生成後にSetup()で各TankManagerを初期化。カメラ追従対象(Transform[])も同時に設定。

```
// GameManager.cs 抜粋
private IEnumerator GameLoop()
{
    yield return StartCoroutine(RoundStarting());
    yield return StartCoroutine(RoundPlaying());
    yield return StartCoroutine(RoundEnding());

    if (m_GameWinner != null)
        SceneManager.LoadScene(0);
    else
        StartCoroutine(GameLoop());
}

// 生存判定
private bool OneTankLeft()
{
    int numTanksLeft = 0;
    for (int i = 0; i < m_PlayerCount; i++)
    {
        if (m_SpawnPoints[i].m_Instance.activeSelf)
            numTanksLeft++;
    }
    return numTanksLeft <= 1;
}
```

GameManager — ゲーム進行フロー図

MainMenu → GameScene（ラウンドループ）



TankMovement — プレイヤー移動

操作・TPS制御・爆風ノックバック・エンジン音

CharacterController移動

Rigidbodyを使わずCharacterControllerで3D移動。stepOffset・slopeLimit・skinWidthを設定して段差・坂を乗り越え、常に水平・地面高さへ補正。

カメラ基準の操作

m_IsDirectControl時は入力をCamera.main基準の前方・右方向へ変換して移動し、移動方向へRotateTowardsで滑らかに旋回。ゲームパッドは右スティックの向きを優先。

爆風ノックバック

AddExplosionForce()が爆発位置との距離・半径から減衰を求めてm_ExplosionVelocityを加算。Lerpで徐々に減衰し、無敵中は無効化。

エンジン音制御

移動入力の有無でm_EngineIdling/m_EngineDrivingを切替。切替ごとにランダムなPitch(±0.2)を加えて自然なエンジン音に。

操作方式（InputReaderの入力を3通りに解釈）

通常タンク操作	前後入力で前進/後退、左右入力で旋回（180°/秒）
ダイレクト操作	入力をカメラ基準の方向に変換し、その向きへ滑らかに旋回
ゲームパッド	左スティックで前後移動＋旋回。右スティックを倒すと、その向きを優先（720°/秒）

```
// TankMovement.cs 抜粋（Move内）
moveDirection =
    camForward * m_MovementInputValue +
    camRight * m_TurnInputValue;
Vector3 velocity = moveDirection * m_Speed;
velocity += m_ExplosionVelocity;
m_CharacterController.Move(
    velocity * Time.deltaTime);
m_ExplosionVelocity = Vector3.Lerp(
    m_ExplosionVelocity, Vector3.zero,
    Time.deltaTime * 5f);
```

TankShooting — 射撃システム

チャージ射撃・放物線計算・特殊弾

チャージ射撃

長押しでm_CurrentLaunchForceが最小5→最大20へ上昇（0.75秒）し、UIスライダーに反映。フルチャージでも自動発射せず、離した瞬間に発射（自動発射はCPUのみ）。

放物線着弾計算

GetProjectilePosition()が充填率から初速を求め、重力下の二次方程式を解いて着弾予測位置を算出。TankAIが射程確認と狙い撃ちに利用。

特殊弾（強化弾）

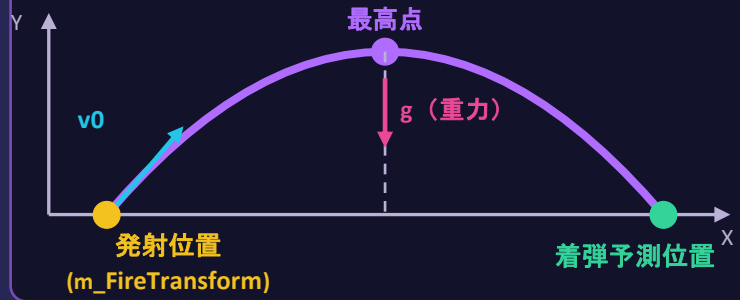
EquipSpecialShell()で倍率を設定し、次の1発のダメージを倍率分だけ強化。発射後は自動解除され、HUDのアクティブ表示も消える。

クールタイム・エイム矢印

発射後m_ShotCooldown(1.0秒)は再チャージ不可で、エイム矢印を赤く表示（フルチャージ中は矢印が振動）。CPUはStopCharging()が同じFire()を呼ぶ。

放物線着弾計算 — GetProjectilePosition()

v0 = Lerp(最小, 最大, 充填率) × 発射方向
a = 0.5 × g.y　b = v0.y　c = 発射高さ
t = (−b ± √(b²−4ac)) / 2a　→ 正の解を採用




```
// TankShooting.cs 抜粋（GetProjectilePosition）
float a = 0.5f * Physics.gravity.y;
float b = velocity.y;
float c = m_FireTransform.position.y;
float answer = answer1 > 0 ? answer1 : answer2;
```

TankAI — CPU制御

NavMesh経路探索・Seek/Flee状態機械・射撃タイミング判定

NavMesh経路探索

ランダム間隔(0.3〜0.6秒)で全敵へのNavMeshPathを計算し、最短経路の敵をターゲットに確定。Cornerを順に追いかけて追跡する。

Seek → Flee 遷移

敵との距離が3m未満の状態が2秒以上続くとFleeへ。逃走先は90〜180度のランダム方向・5〜20mでNavMesh計算。2秒以上進めなければ再計算。

射撃タイミング判定

射程内かつNavMesh.Raycastで壁なし→チャージ開始。着弾予測が敵との距離に届き、dot積>0.99（ほぼ正面）でStopCharging()して発射。

負荷分散

m_PathfindTimeをAwakeでRandom.Range(0.3f, 0.6f)に設定。複数CPUの経路探索を時間的にばらつかせ、フレームスパイクを軽減。

SEEK　追跡（初期状態）

最短経路の敵を追跡（NavMesh.CalculatePath・0.3〜0.6秒ごとに更新）

射程内＆壁なし → チャージ → dot>0.99 で発射

距離<3m が2秒継続

逃走経路の完了

FLEE　逃走

90〜180°のランダム方向・5〜20m先へNavMesh経路を生成

2秒以上進めなければ経路を再計算

```
// TankAI.cs 抜粋（Seek → Flee）
if (targetDistance < 3.0f) {
    m_TimeCloseToTarget += Time.deltaTime;
    if (m_TimeCloseToTarget > 2.0f)
    { StartFleeing(); return; }
} else m_TimeCloseToTarget = 0.0f;
```

TankHealth — HP管理

HP管理・無敵時間・シールド・HPバーアニメーション

HP管理

TakeDamage()がシールド軽減後のダメージでm_CurrentHealthを減算。HP0でOnDeath()を呼び、爆発エフェクト・爆発音→SetActive(false)でラウンド終了を通知。

無敵時間・点滅演出

被弾後にInvincibleRoutine()を起動。m_BlinkInterval(0.1秒)ごとにRenderer.enabledを切り替えて点滅し、m_InvincibleTime(1.5秒)で解除。

シールド

SetShield()で軽減率(0〜1)を設定し、受けるダメージを(1−m_ShieldValue)倍に。PowerUpアイテムから付与・解除でき、倍率も動的に変えられる。

HPバーアニメーション

表示用m_DisplayHealthをLerp(速度8)で実HPへ追従させスライダーを更新。HP割合に応じてm_FullHealthColor→m_ZeroHealthColorへ変色。

```
// TankHealth.cs 抜粋
public void TakeDamage(float amount)
{
    if (m_IsInvincible || m_Dead) return;
    m_CurrentHealth = Mathf.Max(
        m_CurrentHealth -
        amount * (1f - m_ShieldValue), 0f);
    if (m_CurrentHealth <= 0f)
        { OnDeath(); return; }
    if (m_InvincibleCoroutine != null)
        StopCoroutine(m_InvincibleCoroutine);
    m_InvincibleCoroutine =
        StartCoroutine(InvincibleRoutine());
}

// HPバー更新 SetHealthUI()
m_DisplayHealth = Mathf.Lerp(
    m_DisplayHealth, m_CurrentHealth,
    Time.deltaTime * m_HealthBarSmoothSpeed);
m_Slider.value = m_DisplayHealth;
m_FillImage.color = Color.Lerp(
    m_ZeroHealthColor, m_FullHealthColor,
    m_DisplayHealth / m_StartingHealth);
```

RaceGameManager ― レース全体の構成と進行

車種選択 → スタート演出 → レース → 結果発表を、フェーズで一元管理

フェーズ管理

RacePhase（Init / Select / Countdown / Racing / Ending / Results）で状態を一元管理。GOの瞬間にレース時間の計測を始め、全車の操作ロックを解除する。

車の生成と配置

1つのプレハブから最大4台を生成し、P1青・P2赤・P3緑・P4黄に塗り分け。RaceGridが2列のスタート位置を計算し、車種の点数で性能と車体サイズも変える。

画面・UIの組み立て

分割画面カメラ・HUD・ミニマップを人数に合わせて自動で構成。タンクモード時代の旧表示（LapUI・RankUI）は、スクリプトを止めずに表示だけ消す。

参加者の引き継ぎ

車種選択の結果はstaticなRaceSessionに保存し、シーンをまたいで保持。リトライは同じ顔ぶれ・同じ車で再開し、車選びに戻ると前回の顔ぶれが復元される。



CarMovement ― 走行とドリフト

速度を直接制御するマリカー式の走行と、3段階のミニターボ

速度スカラーの直接制御

currentSpeedを目標速度へ指数的に近づけ（フレームレートに依存しない）、毎FixedUpdateでrb.linearVelocityに「前方 × currentSpeed」を代入する。AddForce方式より慣性が出にくく、扱いやすい。

速度帯ごとの旋回

旋回レートは低速×0.5・中速×1.3・高速×1.75と速度帯で変わり、極低速では曲がれない。後退中は操作が逆になる。

ドリフト

Shift / R1を押した瞬間に、最高速の50%以上・接地中なら開始。方向はスティックでロックされ、内側に倒すと旋回×2.1、外側は×0.5。速度が落ちると中断し、離すとゲージの段階に応じたブーストが発動する。

坂・ダート・接地

地面へRayを撃って法線を取り、80°以下の坂では車体をSlerpで法線に合わせる。ダート（Groundタグ）は最高速×0.5。ダウンフォースで接地を保つ。

ミニターボ（ドリフトゲージは毎秒 +60）

段階	ゲージ（目安）	速度	持続
①	60超（約1秒）	×1.25	1秒
②	120超（約2秒）	×1.5	1.25秒
③	240超（約4秒）	×1.75	1.5秒

ドリフトボタンを離すと、到達した段階のブーストが発動。速度が落ちて中断した場合はブーストなし。

```
// CarMovement.cs 抜粋（FixedUpdate）
// 目標速度へ指数追従（FPSに依存しない）
currentSpeed = Mathf.Lerp(currentSpeed, cap,
    1f - Mathf.Exp(-response * dt));

// 向いている方向へ、速度を直接代入
Vector3 vel = transform.forward * currentSpeed;
if (drifting)
    vel += -transform.right * (driftDirection *
        Mathf.Abs(currentSpeed) *
        driftSidewaysFriction * driftSteerRamp);
rb.linearVelocity = vel;
```

車タイプ ― CarSpec / CarTuning

5項目を1〜5点で配分（合計15点）し、点数を走行パラメータへ変換

車種	スピード	かそく	おもさ	まがりやすさ	すべりにくさ	合計	特徴
BALANCE　バランス	3	3	3	3	3	15	クセがなく、走らせやすい
SPEED　スピード	**5**	2	2	2	**4**	15	直線は速いが、加速はゆっくり
DASH　ダッシュ	2	**5**	2	3	3	15	加速がするどい
HEAVY　ヘビー	**4**	1	**5**	2	3	15	重くて、ぶつかり合いに強い
DRIFT　ドリフト	3	3	2	**5**	2	15	よく曲がるが、よく滑る

点数 → 倍率（CarTuning）　3点 ＝ ×1.0（プレハブの値）

項目	反映先	1点	2点	3点	4点	5点
スピード	最高速 maxSpeed	×0.96	×0.98	**×1.00**	×1.02	×1.04
かそく	加速の応答 accelResponse	×0.6	×0.8	**×1.0**	×1.2	×1.4
おもさ	質量 Rigidbody.mass	×0.6	×0.8	**×1.0**	×1.2	×1.4
おもさ	車体サイズ ModelScale	×0.90	×0.95	**×1.00**	×1.05	×1.10
まがりやすさ	旋回力 steerStrength	×0.8	×0.9	**×1.0**	×1.1	×1.2
すべりにくさ	横滑り driftSidewaysFriction	×1.5	×1.25	**×1.0**	×0.75	×0.5
すべりにくさ	ダート耐性 groundSpeedMultiplier	×0.8	×0.9	**×1.0**	×1.1	×1.2

3点が基準（×1.0）

差は小さめに設計し、スピードは±4%まで。最高速の差は周回ごとに積み重なるため。

m_TuningInfluence

補正の強さを1つの値で調整できる。倍率＝max(0.1, 1＋(倍率−1)×影響度)。0にすると全車が同じ性能になる。

周回・順位ロジック — RaceRules / LapCounter

判定をUnity非依存の純C#に分離し、チェックポイントの順番通過で周回を数える

LapProgress（周回判定）

チェックポイントは次の番号を通った時だけ有効。全CPを通ってからゴールを通ると1周完了で、飛ばし・逆走は数えない。進捗は Units＝完了周回×(CP数＋1)＋今周の通過CP数 で表す。

RaceRanking（順位）

①ゴール済み（早い順）②Unitsが大きい順 ③同じなら次のCPまでの水平距離が短い順 ④同点なら番号順。最大4台なので挿入ソートで毎フレーム算出する。

RaceFlow（終了判定）

(人数−1)人がゴールした時点で最下位が確定し、順位が決まったら終了（m_EndWhenRankDecided）。全員ゴールか、1着から60秒（m_TimeoutAfterFirstFinish）でも終了する。

Unityとの接続

LapCounter（車ごと）がLapProgressを内包。CheckPoint / GoalTriggerがOnTriggerEnter（Playerタグ）で通過を渡し、OnLapUpdated / OnFinishedでRaceGameManagerへ通知する。

1周の流れ（チェックポイント5個の例）



飛ばし・逆走 → Ignored　　最終周ゴール → Finished

進捗 Units ＝ 完了周回 × (CP数＋1) ＋ 今周の通過CP数

```
// LapProgress.cs 抜粋
// PassCheckPoint：次の番号だけ数える
if (IsFinished ||
    checkPointNumber != NextCheckPoint ||
    checkPointNumber <= 0)
    return GateResult.Ignored;
m_CheckPointsThisLap++;
// PassGoal：全CP通過後だけ1周
if (IsFinished ||
    m_CheckPointsThisLap < m_TotalCheckPoints)
    return GateResult.Ignored;
m_LapsCompleted++;
m_CheckPointsThisLap = 0;
```

画面分割・追従カメラ

RaceSplitScreenCameraManager / RaceCameraFollow — 人数に合わせて自動構成

分割画面の自動構成

Setup(targets)が人数分のカメラを生成して画面を割り当てる。1人は全画面、2人は左右（上下も可）、3人は2×2で1枠空き（上1＋下2も可）、4人は2×2。

追従カメラ

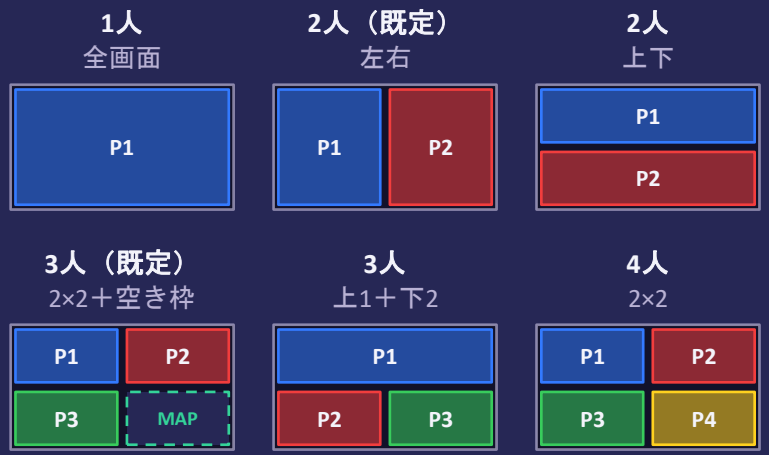
RaceCameraFollowが車の後方（0, 5, −8）から位置と回転をダンピングして追従し、車の向きにも滑らかに合わせる。ターゲットが変わると即座にスナップ。

後ろ振り向き

InputReaderの入力（WASD: C ／ 矢印: 右Ctrl ／ パッド: X・Y）で、押すたびに後方視点へ切替。2人以上では押した人のカメラだけが反転する。

ドリフト段階の揺れ

ミニターボの段階が上がるたびにカメラを揺らす（強さ0.08 / 0.15 / 0.3、0.15秒）。音が二重にならないよう、AudioListenerの重複も防ぐ。



3人のときの空き枠

右下の空き枠にはミニマップを置く（RaceMinimapLayoutのEmptySlot）。塗りつぶしはカメラではなくRaceHudのUI画像で行う（Unity 6.2のURPで、スクリプト生成の分割カメラがUIの上に重なる問題を避けるため）。

HUD・ミニマップ ― AIに生成させたUI

RaceHud / RaceMinimap / RaceUiKit が、分割画面ごとにHUDとミニマップを表示

分割画面ごとのHUD

ビューポートごとにパネルを生成。左上にLAPとタイム、左下に順位、上中央にプレイヤーバッジ（2人以上）。右側の画面は左右を反転し、1人のときは順位を出さない。

演出ポップアップ

「LAP 2」「FINAL LAP!」（最終周はオレンジで脈打つ）、ゴール時の「FINISH!」と順位・タイム、(人数−1)人ゴール後の「最下位確定」帯。順位が変わると数字が一瞬ふくらむ。

ミニマップの自動生成

専用カメラを使わず、上から道のコライダーへRayを撃って走路を検出。フェンス等を除外し、穴埋め→平滑化→ノイズ除去→CP・ゴール・車とつながる範囲の抽出→切り抜きの順に整形してテクスチャ化する。

アイコンとマーカー

各車のアイコンはCarMovementから取得。1人プレイは自車を強調し、複数人ではプレイヤー番号を表示。スタート／ゴールはGoalTriggerの位置を道の中央へ補正して置く。



表示の重なり順（Canvasの sortingOrder）

HUD	車種選択	結果発表	スタート演出
100	200	300	500

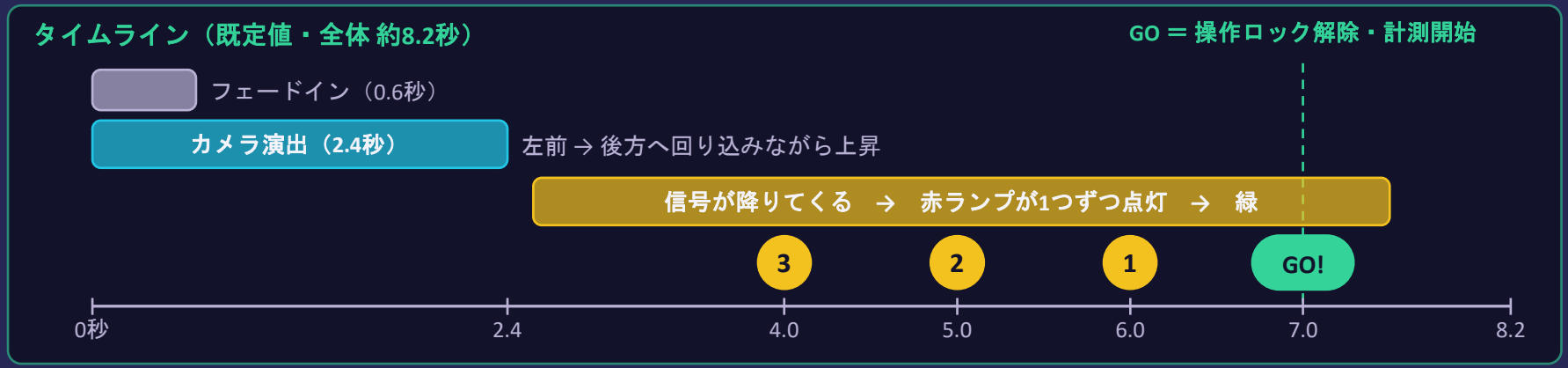
順位表示の色

1st ／ 2nd ／ 3rd ／ 4th（白）

順位が変わると、数字が0.35秒だけふくらんで知らせる。

RaceStartSequence — スタート演出

フェードイン → カメラ演出 → 信号 → カウントダウン → GO!（約8.2秒）



カメラ演出

左前方（−40°）から後方へ回り込みながら上昇し、追従カメラと同じ位置に着地するので、切り替わりで跳ばない。Enter / Startでスキップ可。

信号とカウント

3・2・1で赤ランプが1つずつ点き、ビープ音が鳴る。HUDは「3」でフェードイン。GOで緑に変わり、操作ロック解除・FOVキック（＋8°）・スピード線。

車のロック

演出中はCarMovementを無効化し、Rigidbodyの位置(X/Z)と回転を固定（Yは自由）。GOで再有効化し、速度とドリフトを0に戻す。

失敗しても止まらない

途中で例外が起きても演出を中断し、車・カメラ・HUDを復元してレースを必ず開始する。

車種選択・結果発表

CarSelectMenu / RaceResultsUI — 4人まで、キーボード・パッドで操作

CarSelectMenu — 車種選択（最大4人）



参加 → 車種選択 → READY → 自動スタート

デバイス	決定	キャンセル	左右
WASD	Space	Q	A / D
矢印キー	Enter	Backspace	← / →
ゲームパッド	A	B	十字キー／スティック

3Dプレビュー：車のメッシュを別空間にコピーしてRenderTextureで表示。重さで大きさが変わり、プレイヤー色で塗装する。

RaceResultsUI — 結果発表

1st	P2	DRIFT	1:23.456
2nd	P1	BALANCE	1:25.102
3rd	P3	DASH	未ゴール
4th	P4	SPEED	最下位

RETRY　CAR SELECT　TITLE

※ 表示イメージ（数値は例）

リザルトの動き

- 下位から順に右からスライド表示（0.5秒間隔）。1位の行は金色に発光
- 「未ゴール」は時間切れの人、「最下位」は赤で表示
- RETRY（同じ顔ぶれ）／CAR SELECT／TITLE。左右＋決定かマウスで選択

InputReader — 入力システム

キーボード2系統とゲームパッド複数台を1つの層で吸収（タンク・レース共通）

操作	WASD	矢印キー	ゲームパッド
割り当て（ControlIndex）	1	2	10＋接続順（10以上）
移動・ステアリング	W A S D	矢印キー	左スティック
アクセル（レース）	W	↑	右ボタン（Xbox B／PS ○）
ブレーキ・後退（レース）	S	↓	下ボタン（Xbox A／PS ×）
ドリフト（レース）	左Shift	右Shift	R1
後ろを見る（レース）	C	右Ctrl	X／Y ボタン
発射（タンク）	Space	Enter	R1

ゲームパッドの割り当て（レース）

アクセル／ブレーキはボタン式（前後入力は 1 / −1 / 0）で、左スティックはステアリング専用。右スティックはタンクの向き指定（LookInput）に使う。

ControlIndex

0＝自動判定（WASD・矢印・パッド1を合成。1人テスト用）、1＝WASD、2＝矢印、10以上＝ゲームパッド（10＋接続順）。人ごとの割り当てに使う。

InputLocked

trueの間は全デバイスの入力を中立にする。スタートの合図まで・ゴール後に使い、無効なControlIndexでも毎フレームの警告が出ない。

実行順序 −100

[DefaultExecutionOrder(-100)]でTankMovementなどより先に入力を読み、1フレーム遅れるのを防ぐ。

現状のまとめと今後の予定

5つのゲームモードのうち2つが完成。次は残りのモードとオンライン対応へ

現状

2 / 5 モード完成

- **タンクモード** — ラウンド制（CPUあり） — 完成
- **レースモード** — 最大4人・3周のレース — 完成
- ○ 3つ目のモード — 制作中
- ○ 4つ目のモード — 制作中
- ○ 5つ目のモード — 制作中

今後の予定

01 **残りのゲームモードを制作**
3つ目以降は制作中。内容を話し合いながら、実装を進めていく。

02 **オンラインプレイに対応**
今の遊び方に加えて、オンラインにも対応させる。

03 **プレイヤー自身がモードを選べる**
遊びたいモードを、プレイヤー自身が選べるようにする。

04 **5つのモードをラリー形式で通しプレイ**
5つのモードを順番に通して遊べる、ラリー形式を用意する。

05 **モード単体プレイの試合数を変えられる**
例：タンクモードだけを選んで遊ぶとき、試合数を変更できる。
1試合で終わらず、好きなモードを好きなだけ遊べる。